

Document 7: Human-AI Teaming Framework for Semantic Governance

Integrating Human Factors Across AI-Driven Knowledge Engineering

Version: 1.0

Date: February 2026

Purpose: Close critical human factors gaps identified in Cummings research across Semantic Governance architecture

Executive Summary

The Semantic Governance architecture comprises world-class technical infrastructure: federated RAG 2.0, AI Circuit Breakers, vector databases, comprehensive service portfolios, and operational tooling. However, research by Dr. Missy Cummings and others on human-automation interaction reveals a critical pattern: **technically sophisticated systems fail when human oversight degrades due to automation complacency, alert fatigue, skill atrophy, and trust miscalibration.**

This framework addresses the 20% missing layer: **Human-AI Teaming**. It ensures knowledge engineers can effectively supervise AI agents, maintain decision-making competency, calibrate trust appropriately, and sustain performance over extended operations.

Integration Points

This document integrates with:

- **Semantic Governance Architecture** (Solution Architecture PDF) - adds human factors to federated agents
- **AI Circuit Breaker Framework** (INCOSE paper) - adds human takeover protocols
- **Vector DB/Context Engine** - adds human comprehension interfaces
- **Service Portfolio** - adds human performance services
- **Operating Model** - adds human-centered workflows
- **Tom Sawyer Workbench** - adds cognitive support visualizations

Part 1: Theoretical Foundation

1.1 The Human-Automation Paradox

Cummings' Core Finding:

"The better the automation, the worse the human performance when automation fails or requires oversight."

Why This Matters for Semantic Governance:

Automation Quality Paradox in Knowledge Engineering:

- Week 1: AI suggests mappings with 65% accuracy
- Knowledge engineers carefully review each suggestion
 - Build deep understanding of domain patterns
 - Maintain sharp validation skills

- Week 12: AI suggests mappings with 92% accuracy
- Knowledge engineers trust AI recommendations
 - Review becomes perfunctory ("rubber stamping")
 - Domain knowledge atrophies (don't need to think deeply)
 - When AI fails (8% error rate), humans can't detect failures

Result: Better AI → Worse human supervision → Higher risk

1.2 Critical Human Factors Failures

From Aviation (Cummings), Medical Devices (FDA), Industrial Control (NTSB):

Failure Mode	Manifestation in Semantic Governance	Consequence
Automation Complacency	Accepting high-confidence mappings without verification	Bad ontology mappings propagate to 30,000+ assets
Alert Fatigue	Ignoring "flag_for_manual_review" after seeing 100/day	Critical conflicts auto-accepted inappropriately
Mode Confusion	Not knowing which domain agent is active	Query returns incomplete results, user unaware
Skill Degradation	Loss of manual mapping ability after 6 months of AI automation	Cannot intervene effectively when AI fails

Trust Miscalibration	Treating confidence=0.85 as "reliable" when actual accuracy=0.62	Systematic acceptance of incorrect mappings
Out-of-Loop Performance	Cognitive inability to take over after long monitoring periods	Cannot resolve complex conflicts when flagged

1.3 Design Principles for Human-AI Teaming

Principle 1: Human as Active Participant, Not Passive Monitor

-  Wrong: "Human reviews AI outputs when flagged"
-  Right: "Human periodically performs tasks manually to maintain skill"

Principle 2: Transparency Over Autonomy

-  Wrong: "AI confidence score: 0.87" (opaque)
-  Right: "High confidence because: exact string match + 15 validated precedents + active schema" (explainable)

Principle 3: Calibrated Trust Through Feedback

-  Wrong: "AI accuracy: 89%" (system-level metric)
-  Right: "Your acceptance rate for 0.85 confidence: 94% | Actual accuracy at 0.85: 71% | You are over-trusting" (personalized feedback)

Principle 4: Graded Autonomy, Not Binary

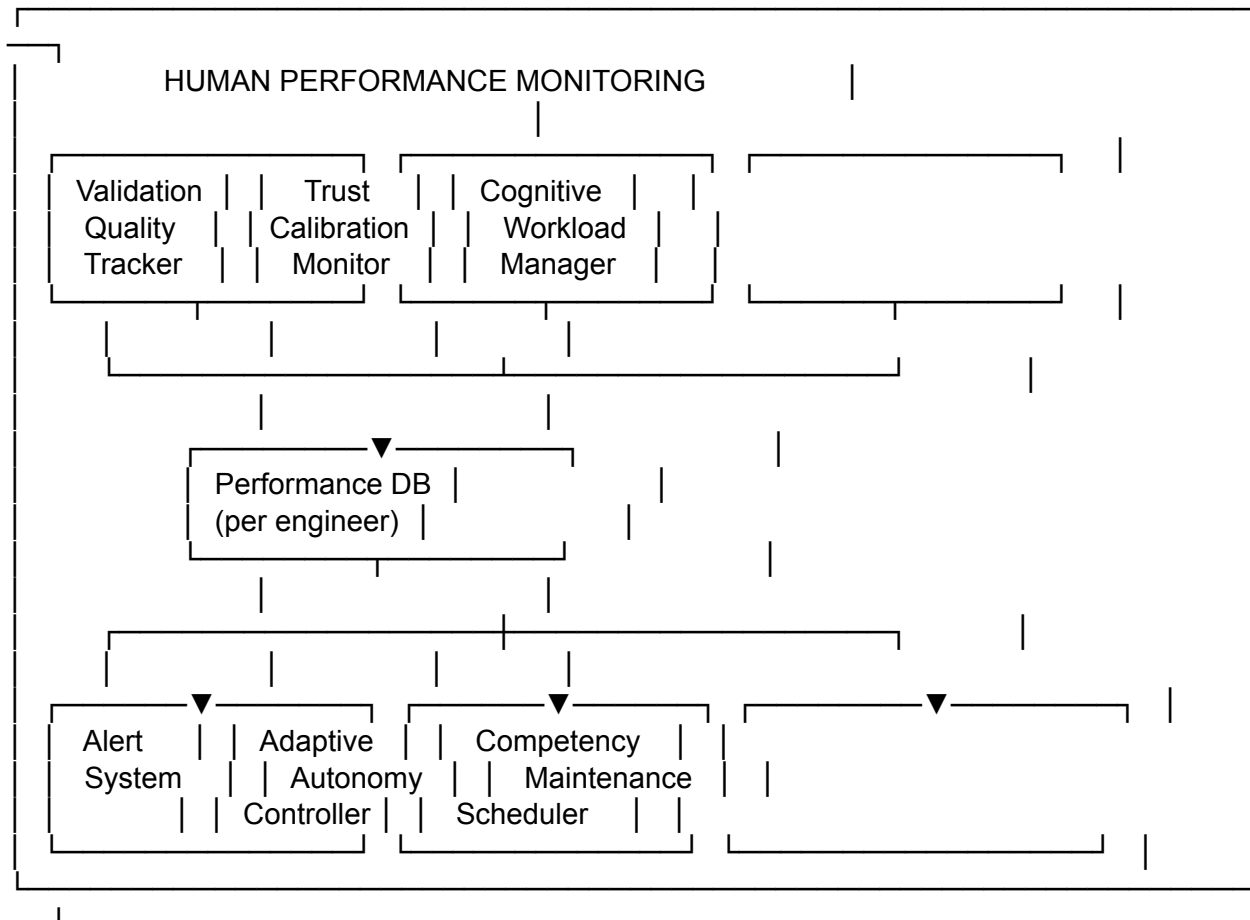
-  Wrong: "Auto-accept >0.80 confidence" OR "Manual review everything"
-  Right: "Autonomy adjusts based on: domain complexity, recent accuracy, human workload, stake level"

Principle 5: Organizational Safety Culture

-  Wrong: "Production pressure to accept 100 mappings/day"
 -  Right: "Quality metrics trump throughput metrics | Authority to halt for safety concerns"
-

Part 2: Human Performance Monitoring System

2.1 Architecture Overview



2.2 Human_Performance_Metrics Schema

-- Core human performance tracking

```
CREATE TABLE Human_Performance_Metrics (  
  metric_id UUID PRIMARY KEY,  
  engineer_id UUID NOT NULL,  
  session_id UUID NOT NULL,  
  timestamp TIMESTAMP NOT NULL,
```

-- Decision Context

```
  decision_type VARCHAR(50), -- 'mapping_validation', 'conflict_resolution', 'manual_mapping'  
  task_complexity FLOAT, -- 0-1 scale  
  ai_confidence FLOAT, -- when AI involved
```

-- Decision Outcome

```
action_taken VARCHAR(50), -- 'accepted', 'rejected', 'modified', 'deferred'
time_to_decision_seconds INT,
decision_correct BOOLEAN, -- determined by subsequent validation
```

```
-- Human State Indicators
session_duration_minutes INT,
decisions_in_session INT,
time_since_last_break_minutes INT,
```

```
-- Calculated Metrics
validation_accuracy_rolling FLOAT, -- last 20 decisions
trust_calibration_score FLOAT, -- -1 to +1 (0 = perfect calibration)
cognitive_load_estimate FLOAT, -- 0-1 scale
fatigue_score FLOAT, -- 0-1 scale
```

```
-- Evidence
reasoning_provided TEXT, -- did engineer document their thinking?
checklist_completion_rate FLOAT, -- 0-1
```

```
FOREIGN KEY (engineer_id) REFERENCES Engineers(id),
FOREIGN KEY (session_id) REFERENCES Work_Sessions(id)
```

```
);
```

```
-- Session-level tracking
CREATE TABLE Work_Sessions (
  session_id UUID PRIMARY KEY,
  engineer_id UUID NOT NULL,
  start_time TIMESTAMP NOT NULL,
  end_time TIMESTAMP,
```

```
-- Session Metrics
total_decisions INT DEFAULT 0,
session_accuracy FLOAT,
avg_time_per_decision FLOAT,
```

```
-- Alerts Triggered
fatigue_alerts INT DEFAULT 0,
over_trust_alerts INT DEFAULT 0,
under_trust_alerts INT DEFAULT 0,
quality_degradation_alerts INT DEFAULT 0,
```

```
-- Interventions
breaks_taken INT DEFAULT 0,
competency_exercises_completed INT DEFAULT 0,
```

```
FOREIGN KEY (engineer_id) REFERENCES Engineers(id)
```

```
);
```

```

-- Trust calibration tracking
CREATE TABLE Trust_Calibration_Events (
  event_id UUID PRIMARY KEY,
  engineer_id UUID NOT NULL,
  timestamp TIMESTAMP NOT NULL,

  -- AI Prediction
  ai_confidence FLOAT NOT NULL,
  ai_recommendation VARCHAR(50),

  -- Human Decision
  human_decision VARCHAR(50),
  human_confidence_estimate FLOAT, -- if collected

  -- Ground Truth (determined later)
  actual_correctness BOOLEAN,

  -- Calibration Analysis
  trust_alignment_score FLOAT, -- how well human trust matched reality
  over_trust_indicator BOOLEAN, -- accepted when shouldn't have
  under_trust_indicator BOOLEAN, -- rejected when shouldn't have

  FOREIGN KEY (engineer_id) REFERENCES Engineers(id)
);

-- Skill maintenance tracking
CREATE TABLE Competency_Assessments (
  assessment_id UUID PRIMARY KEY,
  engineer_id UUID NOT NULL,
  assessment_date TIMESTAMP NOT NULL,
  assessment_type VARCHAR(50), -- 'manual_mapping', 'knowledge_quiz', 'conflict_resolution'

  -- Scores
  accuracy_score FLOAT,
  speed_score FLOAT,
  completeness_score FLOAT,
  overall_score FLOAT,

  -- Comparison to baseline
  baseline_score FLOAT,
  score_delta FLOAT,
  skill_degradation_detected BOOLEAN,

  -- Recommended actions
  training_needed BOOLEAN,
  exercises_recommended TEXT[],

```

```
FOREIGN KEY (engineer_id) REFERENCES Engineers(id)
);
```

2.3 Validation Quality Tracker Service

```
class ValidationQualityTracker:
```

```
    """
```

```
    Monitors quality of human validation decisions over time.
    Detects accuracy degradation, alert fatigue, rubber stamping.
    """
```

```
    def __init__(self, performance_db):
```

```
        self.db = performance_db
```

```
        self.alert_thresholds = {
```

```
            'accuracy_drop': 0.15, # 15% below baseline
```

```
            'speed_increase': 2.0, # 2x faster than baseline (rushing)
```

```
            'fatigue_threshold': 0.7, # 0-1 scale
```

```
            'session_duration_max': 360 # 6 hours
```

```
        }
```

```
    def record_decision(self, engineer_id, session_id, decision_event):
```

```
        """Record a validation decision and calculate real-time metrics."""
```

```
        # Extract decision details
```

```
        task = decision_event['task']
```

```
        action = decision_event['action_taken']
```

```
        time_spent = decision_event['time_to_decision_seconds']
```

```
        # Calculate session metrics
```

```
        session_metrics = self._calculate_session_metrics(engineer_id, session_id)
```

```
        # Detect quality degradation
```

```
        alerts = []
```

```
        # Alert 1: Accuracy declining
```

```
        if session_metrics['accuracy_rolling'] < (session_metrics['baseline_accuracy'] -
```

```
self.alert_thresholds['accuracy_drop']):
```

```
            alerts.append({
```

```
                'type': 'ACCURACY_DEGRADATION',
```

```
                'severity': 'HIGH',
```

```
                'message': f"Validation accuracy declined to {session_metrics['accuracy_rolling']:.1%}
```

```
(baseline: {session_metrics['baseline_accuracy']:.1%})",
```

```
                'recommendation': 'Review recent decisions and consider taking a break'
```

```
            })
```

```

# Alert 2: Rushing through decisions
baseline_time = session_metrics['baseline_decision_time']
if time_spent < (baseline_time / self.alert_thresholds['speed_increase']):
    alerts.append({
        'type': 'RUSHING_DETECTED',
        'severity': 'MEDIUM',
        'message': f"Decision made in {time_spent}s (typical: {baseline_time}s). Ensure
thorough review.",
        'recommendation': 'Slow down and use validation checklist'
    })

# Alert 3: Fatigue indicators
fatigue_score = self._calculate_fatigue_score(session_metrics)
if fatigue_score > self.alert_thresholds['fatigue_threshold']:
    alerts.append({
        'type': 'FATIGUE_DETECTED',
        'severity': 'HIGH',
        'message': f"Fatigue indicators detected (score: {fatigue_score:.2f})",
        'recommendation': 'Take a 15-minute break before continuing'
    })

# Alert 4: Session too long
if session_metrics['session_duration_minutes'] >
self.alert_thresholds['session_duration_max']:
    alerts.append({
        'type': 'SESSION_DURATION_EXCEEDED',
        'severity': 'CRITICAL',
        'message': f"Session duration: {session_metrics['session_duration_minutes']} minutes
(max recommended: {self.alert_thresholds['session_duration_max']})",
        'recommendation': 'End session and resume after rest period'
    })

# Store metrics
self._store_performance_metrics(engineer_id, session_id, decision_event,
session_metrics, alerts)

return {
    'metrics': session_metrics,
    'alerts': alerts
}

def _calculate_session_metrics(self, engineer_id, session_id):
    """Calculate rolling metrics for current session."""

# Get recent decisions (last 20)
recent_decisions = self.db.get_recent_decisions(engineer_id, session_id, limit=20)

```

```

# Calculate accuracy (when ground truth available)
correct_decisions = [d for d in recent_decisions if d['decision_correct'] == True]
total_validated = [d for d in recent_decisions if d['decision_correct'] is not None]
accuracy_rolling = len(correct_decisions) / len(total_validated) if total_validated else None

# Calculate average decision time
avg_decision_time = sum(d['time_to_decision_seconds'] for d in recent_decisions) /
len(recent_decisions)

# Get baseline metrics (engineer's historical average)
baseline = self.db.get_baseline_metrics(engineer_id)

# Get session duration
session = self.db.get_session(session_id)
session_duration = (datetime.now() - session['start_time']).total_seconds() / 60

return {
    'accuracy_rolling': accuracy_rolling,
    'baseline_accuracy': baseline['accuracy'],
    'avg_decision_time': avg_decision_time,
    'baseline_decision_time': baseline['decision_time'],
    'decisions_in_session': len(self.db.get_all_session_decisions(session_id)),
    'session_duration_minutes': session_duration,
    'time_since_last_break': session['time_since_last_break_minutes']
}

def _calculate_fatigue_score(self, session_metrics):
    """
    Calculate fatigue score based on multiple indicators.
    Returns 0-1 (0 = fresh, 1 = exhausted)
    """

    factors = []

    # Factor 1: Session duration (6 hours = 1.0)
    duration_factor = min(session_metrics['session_duration_minutes'] / 360, 1.0)
    factors.append(duration_factor * 0.3) # 30% weight

    # Factor 2: Time since last break (90 minutes = 1.0)
    break_factor = min(session_metrics['time_since_last_break'] / 90, 1.0)
    factors.append(break_factor * 0.3) # 30% weight

    # Factor 3: Accuracy degradation
    if session_metrics['accuracy_rolling'] and session_metrics['baseline_accuracy']:
        accuracy_delta = session_metrics['baseline_accuracy'] -
session_metrics['accuracy_rolling']
        accuracy_factor = max(accuracy_delta / 0.2, 0) # 20% drop = 1.0

```

```

        factors.append(min(accuracy_factor, 1.0) * 0.2) # 20% weight

    # Factor 4: Decision speed increase (rushing indicator)
    speed_ratio = session_metrics['baseline_decision_time'] /
session_metrics['avg_decision_time']
    speed_factor = max((speed_ratio - 1.0) / 1.0, 0) # 2x faster = 1.0
    factors.append(min(speed_factor, 1.0) * 0.2) # 20% weight

    return sum(factors)

def _store_performance_metrics(self, engineer_id, session_id, decision_event, metrics,
alerts):
    """Store performance data and alerts."""

    # Insert into Human_Performance_Metrics
    self.db.insert_performance_metric({
        'engineer_id': engineer_id,
        'session_id': session_id,
        'timestamp': datetime.now(),
        'decision_type': decision_event['task']['type'],
        'task_complexity': decision_event['task'].get('complexity', 0.5),
        'ai_confidence': decision_event['task'].get('ai_confidence'),
        'action_taken': decision_event['action_taken'],
        'time_to_decision_seconds': decision_event['time_to_decision_seconds'],
        'session_duration_minutes': metrics['session_duration_minutes'],
        'decisions_in_session': metrics['decisions_in_session'],
        'validation_accuracy_rolling': metrics['accuracy_rolling'],
        'fatigue_score': self._calculate_fatigue_score(metrics)
    })

    # Store alerts
    for alert in alerts:
        self.db.insert_alert(engineer_id, session_id, alert)

```

2.4 Trust Calibration Monitor Service

```

class TrustCalibrationMonitor:
    """
    Monitors alignment between human trust and AI reliability.
    Detects over-trust (automation complacency) and under-trust.
    """

    def __init__(self, performance_db):
        self.db = performance_db
        self.calibration_thresholds = {
            'over_trust_threshold': 0.15, # Accept rate 15% above accuracy
            'under_trust_threshold': 0.15, # Reject rate 15% above error rate

```

```

        'min_samples': 20 # Minimum decisions for reliable calibration
    }

def analyze_trust_calibration(self, engineer_id, confidence_bucket):
    """
    Analyze how well engineer's trust aligns with AI accuracy
    for a specific confidence bucket (e.g., 0.80-0.85).
    """

    # Get all decisions in this confidence bucket with ground truth
    decisions = self.db.get_validated_decisions_by_confidence(
        engineer_id=engineer_id,
        confidence_min=confidence_bucket['min'],
        confidence_max=confidence_bucket['max'],
        has_ground_truth=True
    )

    if len(decisions) < self.calibration_thresholds['min_samples']:
        return {
            'status': 'INSUFFICIENT_DATA',
            'sample_size': len(decisions),
            'min_required': self.calibration_thresholds['min_samples']
        }

    # Calculate actual AI accuracy in this bucket
    correct_suggestions = [d for d in decisions if d['ai_was_correct']]
    actual_accuracy = len(correct_suggestions) / len(decisions)

    # Calculate human acceptance rate
    accepted_decisions = [d for d in decisions if d['action_taken'] == 'accepted']
    acceptance_rate = len(accepted_decisions) / len(decisions)

    # Calculate calibration score
    # Perfect calibration: acceptance_rate == actual_accuracy
    # Over-trust: acceptance_rate > actual_accuracy (accepting bad suggestions)
    # Under-trust: acceptance_rate < actual_accuracy (rejecting good suggestions)

    calibration_delta = acceptance_rate - actual_accuracy

    # Classify trust state
    if abs(calibration_delta) < 0.05:
        trust_state = 'WELL_CALIBRATED'
        severity = 'NONE'
    elif calibration_delta > self.calibration_thresholds['over_trust_threshold']:
        trust_state = 'OVER_TRUST'
        severity = 'HIGH' if calibration_delta > 0.25 else 'MEDIUM'
    elif calibration_delta < -self.calibration_thresholds['under_trust_threshold']:

```

```

        trust_state = 'UNDER_TRUST'
        severity = 'MEDIUM'
    else:
        trust_state = 'SLIGHTLY_MISCALIBRATED'
        severity = 'LOW'

    # Detailed analysis
    false_accepts = [d for d in decisions if d['action_taken'] == 'accepted' and not
d['ai_was_correct']]
    false_rejects = [d for d in decisions if d['action_taken'] == 'rejected' and d['ai_was_correct']]

    return {
        'status': 'ANALYZED',
        'confidence_bucket': confidence_bucket,
        'sample_size': len(decisions),
        'actual_accuracy': actual_accuracy,
        'acceptance_rate': acceptance_rate,
        'calibration_delta': calibration_delta,
        'trust_state': trust_state,
        'severity': severity,
        'false_accepts': len(false_accepts),
        'false_rejects': len(false_rejects),
        'recommendations': self._generate_calibration_recommendations(
            trust_state, calibration_delta, actual_accuracy
        )
    }

def _generate_calibration_recommendations(self, trust_state, delta, accuracy):
    """Generate actionable recommendations based on trust state."""

    recommendations = []

    if trust_state == 'OVER_TRUST':
        recommendations.append({
            'type': 'BEHAVIOR_ADJUSTMENT',
            'message': f"You are over-trusting AI suggestions in this confidence range. You accept
{delta:.1%} more than you should.",
            'action': 'Increase scrutiny of suggestions in this confidence range. Use validation
checklist.'
        })
        recommendations.append({
            'type': 'AWARENESS',
            'message': f"AI accuracy in this range: {accuracy:.1%}. Not as reliable as it appears.",
            'action': 'Treat this confidence range as requiring careful review, not automatic
acceptance.'
        })

```

```

elif trust_state == 'UNDER_TRUST':
    recommendations.append({
        'type': 'BEHAVIOR_ADJUSTMENT',
        'message': f"You are under-trusting AI suggestions. You reject {-delta:.1%} more than
needed.",
        'action': 'AI is more reliable in this range than you think. Consider accepting more
suggestions after validation.'
    })
    recommendations.append({
        'type': 'EFFICIENCY',
        'message': f"AI accuracy in this range: {accuracy:.1%}. You can trust it more.",
        'action': 'Focus detailed review on lower confidence suggestions.'
    })

elif trust_state == 'WELL_CALIBRATED':
    recommendations.append({
        'type': 'POSITIVE_FEEDBACK',
        'message': f"Excellent calibration. Your trust level matches AI reliability.",
        'action': 'Continue current validation approach for this confidence range.'
    })

return recommendations

def monitor_real_time_calibration(self, engineer_id, decision_event):
    """
    Real-time monitoring of trust calibration during a decision.
    Provides warnings if pattern suggests over/under trust.
    """

    ai_confidence = decision_event['ai_confidence']

    # Get historical calibration for this confidence bucket
    bucket = self._get_confidence_bucket(ai_confidence)
    calibration = self.analyze_trust_calibration(engineer_id, bucket)

    if calibration['status'] != 'ANALYZED':
        return None # Insufficient data

    # If over-trust detected, warn before engineer makes decision
    if calibration['trust_state'] == 'OVER_TRUST':
        return {
            'warning': True,
            'message': f"⚠️ CALIBRATION WARNING: You tend to over-trust AI suggestions in
the {ai_confidence:.2f} confidence range.",
            'details': f"Historical: You accept {calibration['acceptance_rate']:.1%} but actual
accuracy is {calibration['actual_accuracy']:.1%}",

```

```

        'recommendation': 'Please review this suggestion carefully using the validation
        checklist.'
    }

```

```

    return None

```

```

def _get_confidence_bucket(self, confidence):
    """Map confidence score to bucket for analysis."""
    buckets = [
        {'min': 0.90, 'max': 1.00, 'label': 'Very High (0.90-1.00)'},
        {'min': 0.80, 'max': 0.90, 'label': 'High (0.80-0.90)'},
        {'min': 0.70, 'max': 0.80, 'label': 'Medium-High (0.70-0.80)'},
        {'min': 0.60, 'max': 0.70, 'label': 'Medium (0.60-0.70)'},
        {'min': 0.00, 'max': 0.60, 'label': 'Low (<0.60)'}
    ]

    for bucket in buckets:
        if bucket['min'] <= confidence < bucket['max']:
            return bucket

    return buckets[-1] # Default to lowest bucket

```

2.5 Cognitive Workload Manager Service

```

class CognitiveWorkloadManager:

```

```

    """
    Manages cognitive load on knowledge engineers.
    Balances task complexity, workload, and human capacity.
    """

```

```

    def __init__(self, performance_db, task_queue):
        self.db = performance_db
        self.queue = task_queue
        self.workload_thresholds = {
            'max_concurrent_high_complexity': 3,
            'max_session_duration': 360, # 6 hours
            'min_break_duration': 15, # minutes
            'break_frequency': 90 # every 90 minutes
        }

```

```

    def assign_next_task(self, engineer_id):

```

```

        """
        Intelligently assign next task based on:
        - Engineer's current cognitive state
        - Task complexity
        - Workload balance
        - Performance trends

```

```

"""

# Get engineer's current state
state = self._assess_engineer_state(engineer_id)

# If breaks needed, don't assign task
if state['needs_break']:
    return {
        'task': None,
        'recommendation': 'TAKE_BREAK',
        'message': state['break_reason'],
        'break_duration_minutes': self.workload_thresholds['min_break_duration']
    }

# Get available tasks
available_tasks = self.queue.get_pending_tasks()

# Filter tasks by engineer's current capacity
suitable_tasks = self._filter_by_cognitive_capacity(
    available_tasks,
    state['cognitive_capacity']
)

if not suitable_tasks:
    return {
        'task': None,
        'recommendation': 'NO_SUITABLE_TASKS',
        'message': 'All pending tasks exceed your current cognitive capacity. Consider taking
a break.'
    }

# Prioritize tasks
prioritized = self._prioritize_tasks(suitable_tasks, state)

# Assign top priority task
assigned_task = prioritized[0]

# Record assignment
self.db.record_task_assignment(engineer_id, assigned_task['id'])

return {
    'task': assigned_task,
    'recommendation': 'PROCEED',
    'cognitive_load_estimate': assigned_task['complexity'],
    'guidance': self._generate_task_guidance(assigned_task, state)
}

```

```

def _assess_engineer_state(self, engineer_id):
    """Assess engineer's current cognitive state."""

    # Get current session
    session = self.db.get_active_session(engineer_id)

    if not session:
        return {
            'cognitive_capacity': 1.0, # Full capacity at start
            'needs_break': False,
            'fatigue_score': 0.0
        }

    # Calculate session metrics
    session_duration = (datetime.now() - session['start_time']).total_seconds() / 60
    time_since_break = session['time_since_last_break_minutes']
    decisions_made = session['total_decisions']

    # Get performance metrics
    recent_performance = self.db.get_recent_performance(engineer_id, minutes=30)

    # Calculate fatigue
    fatigue_indicators = {
        'duration': min(session_duration / 360, 1.0), # 6 hours = 1.0
        'time_since_break': min(time_since_break / 120, 1.0), # 2 hours = 1.0
        'decision_volume': min(decisions_made / 100, 1.0), # 100 decisions = 1.0
    }

    # Add performance-based indicators
    if recent_performance:
        fatigue_indicators['accuracy_drop'] = max(
            0.3,
            (recent_performance['baseline_accuracy'] - recent_performance['current_accuracy']) /
            0
        )
        fatigue_indicators['speed_increase'] = max(
            1.0,
            (recent_performance['baseline_speed'] / recent_performance['current_speed'] - 1.0) /
            0
        )

    # Weighted average
    weights = {
        'duration': 0.25,
        'time_since_break': 0.25,
        'decision_volume': 0.15,
        'accuracy_drop': 0.20,

```

```

        'speed_increase': 0.15
    }

    fatigue_score = sum(
        fatigue_indicators.get(k, 0) * weights[k]
        for k in weights
    )

    # Determine if break needed
    needs_break = False
    break_reason = None

    if session_duration >= self.workload_thresholds['max_session_duration']:
        needs_break = True
        break_reason = f"Session duration ({session_duration:.0f} min) exceeds maximum ({self.workload_thresholds['max_session_duration']} min)"
    elif time_since_break >= self.workload_thresholds['break_frequency']:
        needs_break = True
        break_reason = f"Time since last break ({time_since_break:.0f} min) exceeds recommended frequency ({self.workload_thresholds['break_frequency']} min)"
    elif fatigue_score > 0.7:
        needs_break = True
        break_reason = f"High fatigue score ({fatigue_score:.2f}). Performance degradation detected."

    # Calculate remaining cognitive capacity (inverse of fatigue)
    cognitive_capacity = max(1.0 - fatigue_score, 0.1) # Minimum 10% capacity

    return {
        'cognitive_capacity': cognitive_capacity,
        'fatigue_score': fatigue_score,
        'needs_break': needs_break,
        'break_reason': break_reason,
        'session_duration_minutes': session_duration,
        'decisions_in_session': decisions_made
    }

def _filter_by_cognitive_capacity(self, tasks, cognitive_capacity):
    """Filter tasks that are within engineer's current capacity."""

    return [
        task for task in tasks
        if task['complexity'] <= cognitive_capacity + 0.1 # Allow 10% margin
    ]

def _prioritize_tasks(self, tasks, engineer_state):
    """

```

Prioritize tasks based on:

- Business priority
- Deadline urgency
- Cognitive load matching
- Learning opportunities

"""

```
for task in tasks:
```

```
    score = 0
```

```
    # Business priority (0-100)
```

```
    score += task.get('business_priority', 50)
```

```
    # Urgency based on deadline
```

```
    if task.get('deadline'):
```

```
        hours_until_deadline = (task['deadline'] - datetime.now()).total_seconds() / 3600
```

```
        if hours_until_deadline < 24:
```

```
            score += 50
```

```
        elif hours_until_deadline < 72:
```

```
            score += 25
```

```
    # Cognitive load matching
```

```
    # Prefer tasks slightly below capacity when fatigued
```

```
    # Prefer challenging tasks when fresh
```

```
    capacity = engineer_state['cognitive_capacity']
```

```
    task_complexity = task['complexity']
```

```
    if capacity > 0.8: # Fresh - prefer challenging work
```

```
        if task_complexity > 0.7:
```

```
            score += 30
```

```
    elif capacity < 0.5: # Fatigued - prefer simple work
```

```
        if task_complexity < 0.4:
```

```
            score += 30
```

```
    else: # Moderate - prefer matching complexity
```

```
        match_score = 30 * (1 - abs(capacity - task_complexity))
```

```
        score += match_score
```

```
    # Variety (avoid monotony)
```

```
    recent_task_types = self.db.get_recent_task_types(engineer_state.get('engineer_id'))
```

```
    if task['type'] not in recent_task_types[-5:]: # Last 5 tasks
```

```
        score += 15
```

```
    task['priority_score'] = score
```

```
return sorted(tasks, key=lambda t: t['priority_score'], reverse=True)
```

```
def _generate_task_guidance(self, task, engineer_state):
```

```

"""Generate contextual guidance for task execution."""

guidance = []

# Fatigue warnings
if engineer_state['fatigue_score'] > 0.5:
    guidance.append({
        'type': 'WARNING',
        'message': 'Fatigue detected. Please proceed carefully and use validation checklists.',
        'icon': '⚠️'
    })

# Complexity warnings
if task['complexity'] > 0.7:
    guidance.append({
        'type': 'GUIDANCE',
        'message': 'This is a complex task. Consider reviewing relevant documentation before proceeding.',
        'icon': '📖'
    })

# First-time task type
task_history = self.db.get_task_type_history(
    engineer_state.get('engineer_id'),
    task['type']
)
if not task_history:
    guidance.append({
        'type': 'INFO',
        'message': f'This is your first {task["type"]} task. Reference materials available in help panel.',
        'icon': 'ℹ️'
    })

return guidance

def record_break(self, engineer_id, duration_minutes):
    """Record that engineer took a break."""

    session = self.db.get_active_session(engineer_id)
    if session:
        self.db.update_session_break(session['session_id'], duration_minutes)

    return {
        'status': 'BREAK_RECORDED',

```

```

        'duration': duration_minutes,
        'recommendation': 'Resume work when ready. Cognitive capacity should be
refreshed.'
    }

```

Part 3: Enhanced AI Interfaces with Human Factors

3.1 Confidence Score Explainability Service

```

class ConfidenceExplainer:
    """
    Translates opaque AI confidence scores into human-understandable explanations.
    Provides context, evidence, concerns, and validation guidance.
    """

    def __init__(self, knowledge_graph, vector_db, historical_db):
        self.kg = knowledge_graph
        self.vector_db = vector_db
        self.historical_db = historical_db

    def explain_mapping_confidence(self, mapping_id, confidence_score, engineer_id=None):
        """
        Generate comprehensive explanation of mapping confidence.
        Includes: score interpretation, breakdown, evidence, concerns, validation checklist.
        """

        # Get mapping details
        mapping = self.kg.get_mapping(mapping_id)

        # 1. Score Interpretation (relative to domain)
        interpretation = self._interpret_score_relative(
            confidence_score,
            mapping['source_domain'],
            mapping['target_domain']
        )

        # 2. Confidence Breakdown (what contributes to score)
        breakdown = self._decompose_confidence(mapping, confidence_score)

        # 3. Supporting Evidence
        evidence = self._collect_supporting_evidence(mapping)

        # 4. Potential Concerns
        concerns = self._identify_concerns(mapping)

```

```

# 5. Historical Calibration
calibration = self._get_historical_calibration(confidence_score, mapping["type"])

# 6. Personalized context (if engineer provided)
personal_context = None
if engineer_id:
    personal_context = self._get_personal_context(engineer_id, confidence_score)

# 7. Validation Checklist
checklist = self._generate_validation_checklist(mapping, concerns)

return {
    'confidence_score': confidence_score,
    'interpretation': interpretation,
    'breakdown': breakdown,
    'evidence': evidence,
    'concerns': concerns,
    'historical_calibration': calibration,
    'personal_context': personal_context,
    'validation_checklist': checklist,
    'recommendation': self._generate_recommendation(
        confidence_score, concerns, calibration
    )
}

def _interpret_score_relative(self, score, source_domain, target_domain):
    """
    Interpret confidence score relative to domain-specific context.
    """

    # Get score distribution for this domain pair
    domain_scores = self.historical_db.get_score_distribution(
        source_domain, target_domain
    )

    # Calculate percentile
    percentile = self._calculate_percentile(score, domain_scores)

    # Classify
    if percentile >= 85:
        category = "Very High"
        color = "green"
    elif percentile >= 70:
        category = "High"
        color = "blue"
    elif percentile >= 50:

```

```

        category = "Moderate"
        color = "yellow"
    elif percentile >= 30:
        category = "Low"
        color = "orange"
    else:
        category = "Very Low"
        color = "red"

    return {
        'score': score,
        'category': category,
        'percentile': percentile,
        'color': color,
        'description': f"{category} confidence for {source_domain} → {target_domain} mappings
(top {100-percentile:.0f}%)",
        'context': f"Typical scores for this domain pair range from {domain_scores['p25']:.2f} to
{domain_scores['p75']:.2f}"
    }

def _decompose_confidence(self, mapping, overall_score):
    """
    Break down confidence score into contributing factors.
    """

    # Get component scores from vector DB
    components = self.vector_db.get_mapping_components(mapping['mapping_id'])

    breakdown = {
        'string_similarity': {
            'score': components.get('string_similarity', 0),
            'weight': 0.25,
            'explanation': 'Lexical similarity between concept names',
            'details': f"{mapping['source_concept']} vs '{mapping['target_concept']}"
        },
        'semantic_similarity': {
            'score': components.get('semantic_similarity', 0),
            'weight': 0.30,
            'explanation': 'Vector embedding similarity in semantic space',
            'details': f"Cosine similarity: {components.get('cosine_distance', 0):.3f}"
        },
        'structural_alignment': {
            'score': components.get('structural_similarity', 0),
            'weight': 0.20,
            'explanation': 'Schema structure and relationship patterns',

```

```

        'details': f"Properties aligned: {components.get('property_overlap', 0)}/
{components.get('total_properties', 0)}"
    },
    'historical_precedent': {
        'score': components.get('precedent_score', 0),
        'weight': 0.15,
        'explanation': 'Similar validated mappings exist',
        'details': f"{components.get('similar_mappings', 0)} similar mappings with
{components.get('precedent_accuracy', 0):.1%} accuracy"
    },
    'domain_usage': {
        'score': components.get('usage_score', 0),
        'weight': 0.10,
        'explanation': 'Frequency and recency of schema usage',
        'details': f"Last modified: {components.get('last_modified_date')}, Query frequency:
{components.get('query_frequency', 'unknown')}"
    }
}

```

Verify weighted sum approximates overall score

```

calculated_score = sum(
    comp['score'] * comp['weight']
    for comp in breakdown.values()
)

```

```
breakdown['calculated_total'] = calculated_score
```

```
breakdown['published_total'] = overall_score
```

```
breakdown['explanation_gap'] = overall_score - calculated_score
```

```
return breakdown
```

```
def _collect_supporting_evidence(self, mapping):
```

```
    """Collect specific evidence supporting the mapping."""
```

```
evidence = []
```

```
# Evidence 1: Exact matches
```

```
if mapping.get('exact_name_match'):
```

```
    evidence.append({
        'type': 'EXACT_MATCH',
        'strength': 'STRONG',
        'description': f"Exact string match on field name: '{mapping['matching_field']}'"
    })

```

```
# Evidence 2: Synonym matches
```

```
synonyms = self.kg.get_synonyms(mapping['source_concept'])
```

```
if mapping['target_concept'] in synonyms:
```

```

        evidence.append({
            'type': 'SYNONYM_MATCH',
            'strength': 'STRONG',
            'description': f"'{mapping['target_concept']}' is a known synonym of
'{mapping['source_concept']}"
        })

# Evidence 3: Similar validated mappings
similar_mappings = self.historical_db.get_similar_validated_mappings(
    mapping['source_concept'],
    limit=5
)
if similar_mappings:
    evidence.append({
        'type': 'PRECEDENT',
        'strength': 'MODERATE',
        'description': f"{len(similar_mappings)} similar mappings previously validated",
        'examples': [
            f"{m['source']} → {m['target']} (validated {m['validation_date']})"
            for m in similar_mappings[:3]
        ]
    })

# Evidence 4: Schema metadata alignment
source_meta = self.kg.get_concept_metadata(mapping['source_concept'])
target_meta = self.kg.get_schema_metadata(mapping['target_schema'])

if source_meta.get('data_type') == target_meta.get('data_type'):
    evidence.append({
        'type': 'DATA_TYPE_MATCH',
        'strength': 'MODERATE',
        'description': f"Data types align: {source_meta['data_type']}"
    })

# Evidence 5: Usage patterns
if target_meta.get('usage_context'):
    usage_alignment = self._check_usage_alignment(
        source_meta.get('typical_usage'),
        target_meta['usage_context']
    )
    if usage_alignment > 0.7:
        evidence.append({
            'type': 'USAGE_PATTERN',
            'strength': 'MODERATE',
            'description': f"Schema usage aligns with concept purpose ({usage_alignment:.0%}
match)"
        })

```

```
return evidence
```

```
def _identify_concerns(self, mapping):
    """Identify potential concerns or red flags."""

    concerns = []

    # Concern 1: Schema metadata issues
    target_meta = self.kg.get_schema_metadata(mapping['target_schema'])

    if 'test' in target_meta.get('table_name', '').lower():
        concerns.append({
            'type': 'TEST_SCHEMA',
            'severity': 'HIGH',
            'description': 'Table name contains "test" - may not be production schema',
            'impact': 'Mapping to test schema could cause downstream data quality issues'
        })

    if target_meta.get('last_modified'):
        days_since_modified = (datetime.now() - target_meta['last_modified']).days
        if days_since_modified > 365:
            concerns.append({
                'type': 'STALE_SCHEMA',
                'severity': 'MEDIUM',
                'description': f'Schema not modified in {days_since_modified} days',
                'impact': 'May be deprecated or unused'
            })

    # Concern 2: Missing relationships
    if not target_meta.get('foreign_keys'):
        concerns.append({
            'type': 'NO_RELATIONSHIPS',
            'severity': 'MEDIUM',
            'description': 'No foreign key relationships detected',
            'impact': 'Schema may be isolated or improperly designed'
        })

    # Concern 3: Ambiguous mappings
    alternative_targets = self.vector_db.get_similar_schemas(
        mapping['source_concept'],
        threshold=0.75,
        exclude=[mapping['target_schema']]
    )
    if len(alternative_targets) > 3:
        concerns.append({
            'type': 'MULTIPLE_CANDIDATES',
```

```

        'severity': 'MEDIUM',
        'description': f'{len(alternative_targets)} other schemas have similar names/properties',
        'impact': 'Ambiguity in correct target selection',
        'alternatives': [t['schema_name'] for t in alternative_targets[:3]]
    })

# Concern 4: Data type mismatches
source_meta = self.kg.get_concept_metadata(mapping['source_concept'])
if source_meta.get('data_type') != target_meta.get('data_type'):
    concerns.append({
        'type': 'DATA_TYPE_MISMATCH',
        'severity': 'HIGH',
        'description': f"Data type mismatch: {source_meta.get('data_type')} vs
{target_meta.get('data_type')}",
        'impact': 'May require data transformation or indicate incorrect mapping'
    })

# Concern 5: Confidence drift
historical_accuracy = self.historical_db.get_confidence_accuracy(
    mapping['confidence_score'],
    mapping['type'],
    time_window_days=30
)
if historical_accuracy and historical_accuracy < mapping['confidence_score'] - 0.15:
    concerns.append({
        'type': 'CONFIDENCE_DRIFT',
        'severity': 'HIGH',
        'description': f"Confidence score ({mapping['confidence_score']:.2f}) overstates
historical accuracy ({historical_accuracy:.2f})",
        'impact': 'AI may be overconfident - exercise extra caution'
    })

return concerns

def _get_historical_calibration(self, confidence_score, mapping_type):
    """Get historical accuracy for this confidence level."""

    # Get confidence bucket
    bucket = self._get_confidence_bucket(confidence_score)

    # Get historical performance
    historical = self.historical_db.get_accuracy_by_confidence(
        confidence_min=bucket['min'],
        confidence_max=bucket['max'],
        mapping_type=mapping_type,
        time_window_days=90 # Last 90 days
    )

```

```

if not historical or historical['sample_size'] < 20:
    return {
        'status': 'INSUFFICIENT_DATA',
        'message': 'Not enough historical data to assess calibration'
    }

# Calculate calibration
calibration_error = confidence_score - historical['actual_accuracy']

if abs(calibration_error) < 0.05:
    calibration_status = 'WELL_CALIBRATED'
elif calibration_error > 0.15:
    calibration_status = 'OVERCONFIDENT'
elif calibration_error < -0.15:
    calibration_status = 'UNDERCONFIDENT'
else:
    calibration_status = 'SLIGHT_MISCALIBRATION'

return {
    'status': 'ANALYZED',
    'confidence_bucket': bucket['label'],
    'sample_size': historical['sample_size'],
    'actual_accuracy': historical['actual_accuracy'],
    'calibration_error': calibration_error,
    'calibration_status': calibration_status,
    'trend': historical.get('accuracy_trend', 'STABLE'), # IMPROVING, DECLINING, STABLE
    'interpretation': self._interpret_calibration(calibration_status, calibration_error, historical)
}

def _get_personal_context(self, engineer_id, confidence_score):
    """Get engineer's personal history with this confidence level."""

    # Get engineer's trust calibration for this confidence bucket
    bucket = self._get_confidence_bucket(confidence_score)

    personal_stats = self.historical_db.get_engineer_calibration(
        engineer_id=engineer_id,
        confidence_min=bucket['min'],
        confidence_max=bucket['max']
    )

    if not personal_stats or personal_stats['decisions'] < 10:
        return {
            'status': 'LIMITED_HISTORY',
            'message': 'Limited personal history at this confidence level'
        }

```

```

    return {
        'status': 'AVAILABLE',
        'decisions_made': personal_stats['decisions'],
        'acceptance_rate': personal_stats['acceptance_rate'],
        'accuracy_when_accepted': personal_stats['accuracy_when_accepted'],
        'trust_state': personal_stats['trust_state'], # OVER_TRUST, UNDER_TRUST,
CALIBRATED
        'warning': self._generate_personal_warning(personal_stats)
    }

def _generate_validation_checklist(self, mapping, concerns):
    """Generate actionable validation checklist based on mapping and concerns."""

    checklist = []

    # Standard checks for all mappings
    checklist.append({
        'category': 'BASIC_VALIDATION',
        'items': [
            f"Verify '{mapping['target_schema']}' is a production (not test) schema",
            f"Confirm data types are compatible: {mapping.get('source_data_type')} →
{mapping.get('target_data_type')}",
            "Check that concept meaning aligns with schema purpose"
        ]
    })

    # Concern-specific checks
    if concerns:
        concern_checks = []

        for concern in concerns:
            if concern['type'] == 'TEST_SCHEMA':
                concern_checks.append("⚠️ CRITICAL: Verify this is not a test/dev schema before
accepting")
            elif concern['type'] == 'MULTIPLE_CANDIDATES':
                concern_checks.append(f"Review alternative schemas: {'',
'.join(concern.get('alternatives', []))}")
            elif concern['type'] == 'DATA_TYPE_MISMATCH':
                concern_checks.append("⚠️ CRITICAL: Resolve data type mismatch or confirm
transformation exists")
            elif concern['type'] == 'CONFIDENCE_DRIFT':
                concern_checks.append("⚠️ Extra scrutiny needed - AI confidence may be
inflated")

```

```

    if concern_checks:
        checklist.append({
            'category': 'CONCERN_RESOLUTION',
            'items': concern_checks
        })

# Relationship checks
checklist.append({
    'category': 'RELATIONSHIP_VALIDATION',
    'items': [
        "Verify foreign key relationships exist and make sense",
        "Check if schema is used by downstream systems (impact analysis)",
        "Confirm schema ownership/stewardship is clear"
    ]
})

return checklist

def _generate_recommendation(self, confidence_score, concerns, calibration):
    """Generate overall recommendation for engineer."""

    # High severity concerns = manual review required
    high_severity_concerns = [c for c in concerns if c['severity'] == 'HIGH']

    if high_severity_concerns:
        return {
            'action': 'CAREFUL_REVIEW_REQUIRED',
            'icon': '⚠️',
            'message': f"High-severity concerns detected ({len(high_severity_concerns)}).  
Thorough manual validation required.",
            'confidence_override': 'Do not rely solely on confidence score'
        }

    # Calibration warnings
    if calibration['status'] == 'ANALYZED' and calibration['calibration_status'] == 'OVERCONFIDENT':
        return {
            'action': 'VERIFY_CAREFULLY',
            'icon': '⚠️',
            'message': f"AI is overconfident at this level. Actual accuracy: {calibration['actual_accuracy']:.1%} (vs confidence: {confidence_score:.1%})",
            'confidence_override': f"Treat as ~{calibration['actual_accuracy']:.0%} confidence, not {confidence_score:.0%}"
        }

    # Standard recommendations based on confidence

```

```

if confidence_score >= 0.90:
    return {
        'action': 'ACCEPT_WITH_CHECKLIST',
        'icon': '✓',
        'message': 'Very high confidence. Review validation checklist and accept if all checks
pass.',
        'confidence_override': None
    }
elif confidence_score >= 0.75:
    return {
        'action': 'REVIEW_AND_VALIDATE',
        'icon': '📋',
        'message': 'High confidence. Complete validation checklist before accepting.',
        'confidence_override': None
    }
elif confidence_score >= 0.60:
    return {
        'action': 'DETAILED_REVIEW',
        'icon': '🔍',
        'message': 'Moderate confidence. Conduct detailed review of evidence and
concerns.',
        'confidence_override': None
    }
else:
    return {
        'action': 'MANUAL_VALIDATION',
        'icon': '👉',
        'message': 'Low confidence. Consider manual investigation or request expert review.',
        'confidence_override': None
    }

def _get_confidence_bucket(self, score):
    """Helper to categorize confidence scores."""
    buckets = [
        {'min': 0.90, 'max': 1.00, 'label': 'Very High (0.90-1.00)'},
        {'min': 0.80, 'max': 0.90, 'label': 'High (0.80-0.90)'},
        {'min': 0.70, 'max': 0.80, 'label': 'Medium-High (0.70-0.80)'},
        {'min': 0.60, 'max': 0.70, 'label': 'Medium (0.60-0.70)'},
        {'min': 0.00, 'max': 0.60, 'label': 'Low (<0.60)'}
    ]

    for bucket in buckets:
        if bucket['min'] <= score < bucket['max']:
            return bucket

```

```

return buckets[-1]

def _interpret_calibration(self, status, error, historical):
    """Generate human-readable calibration interpretation."""

    if status == 'WELL_CALIBRATED':
        return f"✓ AI confidence is well-calibrated. When AI shows
{historical['actual_accuracy']:.0%} confidence, it's correct {historical['actual_accuracy']:.0%} of
the time."

    elif status == 'OVERCONFIDENT':
        return f"⚠ AI is overconfident by {error:.0%}. Actual accuracy is
{historical['actual_accuracy']:.0%}, not {historical['actual_accuracy'] + error:.0%}. Exercise
caution."

    elif status == 'UNDERCONFIDENT':
        return f"AI is underconfident by {-error:.0%}. Actual accuracy is
{historical['actual_accuracy']:.0%}, better than confidence suggests. You can trust it more."

    else:
        return f"Slight miscalibration detected ({error:+.0%}). Be aware confidence may not
perfectly reflect accuracy."

def _generate_personal_warning(self, personal_stats):
    """Generate warning if engineer shows trust miscalibration."""

    if personal_stats['trust_state'] == 'OVER_TRUST':
        return {
            'severity': 'WARNING',
            'message': f"⚠ You tend to over-trust AI at this confidence level. You accept
{personal_stats['acceptance_rate']:.0%} but accuracy is only
{personal_stats['accuracy_when_accepted']:.0%}.",
            'recommendation': 'Be more skeptical. Use validation checklist thoroughly.'
        }

    elif personal_stats['trust_state'] == 'UNDER_TRUST':
        return {
            'severity': 'INFO',
            'message': f"You tend to under-trust AI at this level. Acceptance rate:
{personal_stats['acceptance_rate']:.0%}, Accuracy:
{personal_stats['accuracy_when_accepted']:.0%}.",
            'recommendation': 'Consider accepting more suggestions after validation.'
        }

    return None

```

Part 4: Enhanced User Interfaces

4.1 Mapping Validation Interface (React Component)

```
/**
 * Enhanced mapping validation interface with human factors design.
 * Integrates confidence explainability, trust calibration, cognitive support.
 */

import React, { useState, useEffect } from 'react';
import { ConfidenceExplainer } from '../services/ConfidenceExplainer';
import { TrustCalibrationMonitor } from '../services/TrustCalibrationMonitor';
import { ValidationQualityTracker } from '../services/ValidationQualityTracker';

const MappingValidationInterface = ({ mapping, engineerId, sessionId }) => {
  const [explanation, setExplanation] = useState(null);
  const [trustWarning, setTrustWarning] = useState(null);
  const [checklistState, setChecklistState] = useState({});
  const [showingDetails, setShowingDetails] = useState(false);

  useEffect(() => {
    // Load confidence explanation
    const explainer = new ConfidenceExplainer();
    const explained = explainer.explain_mapping_confidence(
      mapping.id,
      mapping.confidence,
      engineerId
    );
    setExplanation(explained);

    // Check for trust calibration warning
    const trustMonitor = new TrustCalibrationMonitor();
    const warning = trustMonitor.monitor_real_time_calibration(
      engineerId,
      { ai_confidence: mapping.confidence }
    );
    setTrustWarning(warning);
  }, [mapping, engineerId]);

  const handleDecision = async (action) => {
    // Record decision for performance tracking
    const tracker = new ValidationQualityTracker();
```

```

const startTime = Date.now();

// ... handle the actual decision (accept/reject/modify)

const timeSpent = (Date.now() - startTime) / 1000;

const result = await tracker.record_decision(engineerId, sessionId, {
  task: {
    type: 'mapping_validation',
    complexity: calculateComplexity(mapping),
    ai_confidence: mapping.confidence
  },
  action_taken: action,
  time_to_decision_seconds: timeSpent,
  checklist_completed: Object.values(checklistState).every(v => v === true),
  reasoning_provided: document.getElementById('reasoning-field')?.value || null
});

// Show alerts if performance issues detected
if (result.alerts && result.alerts.length > 0) {
  showPerformanceAlerts(result.alerts);
}
};

if (!explanation) return <div>Loading...</div>;

return (
  <div className="mapping-validation-container">

    {/* Trust Calibration Warning Banner */}
    {trustWarning && trustWarning.warning && (
      <div className="alert alert-warning trust-warning">
        <div className="alert-icon">⚠️</div>
        <div className="alert-content">
          <strong>{trustWarning.message}</strong>
          <p>{trustWarning.details}</p>
          <p><em>{trustWarning.recommendation}</em></p>
        </div>
      </div>
    )}

    {/* Mapping Header */}
    <div className="mapping-header">
      <h2>Mapping Validation</h2>
      <div className="mapping-summary">
        <span className="source-concept">{mapping.sourceConcept}</span>
      </div>
    </div>
  </div>
);

```

```

    <span className="arrow">→</span>
    <span className="target-schema">{mapping.targetSchema}</span>
  </div>
</div>

{/* Confidence Score with Interpretation */}
<div className="confidence-section">
  <div className="confidence-score-display">
    <div
      className={`confidence-badge confidence-${explanation.interpretation.color}`}
    >
      {(explanation.confidence_score * 100).toFixed(0)}%
    </div>
    <div className="confidence-interpretation">
      <strong>{explanation.interpretation.category} Confidence</strong>
      <p>{explanation.interpretation.description}</p>
      <p className="context-note">{explanation.interpretation.context}</p>
    </div>
  </div>

  {/* Historical Calibration Warning */}
  {explanation.historical_calibration.status === 'ANALYZED' &&
    explanation.historical_calibration.calibration_status !== 'WELL_CALIBRATED' && (
    <div className="calibration-warning">
      <div className="warning-icon">⚠️</div>
      <div className="warning-text">
        {explanation.historical_calibration.interpretation}
      </div>
    </div>
  )}

  {/* Personal Context Warning */}
  {explanation.personal_context?.warning && (
    <div className="personal-warning">
      <div className="warning-icon">
        {explanation.personal_context.warning.severity === 'WARNING' ? '⚠️' : 'ℹ️'}
      </div>
      <div className="warning-content">
        <p>{explanation.personal_context.warning.message}</p>
        <p><em>{explanation.personal_context.warning.recommendation}</em></p>
      </div>
    </div>
  )}

  {/* Expandable Confidence Breakdown */}

```

```

<button
  className="detail-toggle"
  onClick={() => setShowingDetails(!showingDetails)}
>
  {showingDetails ? '▼' : '►'} Why {(explanation.confidence_score * 100).toFixed(0)}%?
</button>

{showingDetails && (
  <div className="confidence-breakdown">
    <h4>Confidence Score Breakdown</h4>
    <table className="breakdown-table">
      <thead>
        <tr>
          <th>Factor</th>
          <th>Score</th>
          <th>Weight</th>
          <th>Contribution</th>
          <th>Details</th>
        </tr>
      </thead>
      <tbody>
        {Object.entries(explanation.breakdown).map(([key, component]) => {
          if (typeof component !== 'object') return null;
          return (
            <tr key={key}>
              <td>{component.explanation}</td>
              <td>
                <ProgressBar value={component.score} color="blue" />
                {(component.score * 100).toFixed(0)}%
              </td>
              <td>{(component.weight * 100).toFixed(0)}%</td>
              <td>
                <ProgressBar value={component.score * component.weight} color="green" />
                {(component.score * component.weight * 100).toFixed(0)}%
              </td>
              <td className="detail-text">{component.details}</td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
)}
</div>

{/* Evidence Section */}
<div className="evidence-section">

```

```

<h3>Supporting Evidence</h3>
{explanation.evidence.length > 0 ? (
  <ul className="evidence-list">
    {explanation.evidence.map((item, idx) => (
      <li key={idx} className={`evidence-item strength-${item.strength.toLowerCase()}`}>
        <div className="evidence-badge">{item.strength}</div>
        <div className="evidence-content">
          <strong>{item.description}</strong>
          {item.examples && (
            <ul className="evidence-examples">
              {item.examples.map((ex, i) => (
                <li key={i}>{ex}</li>
              ))}
            </ul>
          )}
        </div>
      </li>
    ))}
  </ul>
) : (
  <p className="no-evidence">No strong supporting evidence found. Extra caution
recommended.</p>
)}
</div>

/* Concerns Section */
{explanation.concerns.length > 0 && (
  <div className="concerns-section">
    <h3>⚠️ Potential Concerns</h3>
    <ul className="concerns-list">
      {explanation.concerns.map((concern, idx) => (
        <li key={idx} className={`concern-item severity-${concern.severity.toLowerCase()}`}>
          <div className="concern-header">
            <span className="concern-badge">{concern.severity}</span>
            <strong>{concern.description}</strong>
          </div>
          <p className="concern-impact">Impact: {concern.impact}</p>
          {concern.alternatives && (
            <p className="concern-alternatives">
              Alternatives to consider: {concern.alternatives.join(', ')}
            </p>
          )}
        </li>
      ))}
    </ul>
  </div>
)

```

```

    })

    { /* Validation Checklist */ }
    <div className="checklist-section">
      <h3>Validation Checklist</h3>
      <p className="checklist-instruction">
        {explanation.recommendation.icon} {explanation.recommendation.message}
      </p>

      {explanation.validation_checklist.map((category, catIdx) => (
        <div key={catIdx} className="checklist-category">
          <h4>{category.category.replace(/_/g, ' ')}</h4>
          <ul className="checklist-items">
            {category.items.map((item, itemIdx) => {
              const checkId = `check-${catIdx}-${itemIdx}`;
              return (
                <li key={itemIdx}>
                  <label className="checkbox-label">
                    <input
                      type="checkbox"
                      id={checkId}
                      checked={checklistState[checkId] || false}
                      onChange={(e) => setChecklistState({
                        ...checklistState,
                        [checkId]: e.target.checked
                      })}
                    />

                    <span className={item.startsWith('⚠') ? 'critical-check' : ''}>
                      {item}
                    </span>
                  </label>
                </li>
              );
            })}
          </ul>
        </div>
      ))}

      <div className="checklist-status">
        <ProgressBar
          value={Object.values(checklistState).filter(v => v).length /
getTotalChecklistItems(explanation)}
          color="green"
        />
      </div>
    </div>
  </div>

```

```

        {Object.values(checklistState).filter(v => v).length} /
        {getTotalChecklistItems(explanation)} checks completed
      </span>
    </div>
  </div>

```

```

    {/* Reasoning Field (Optional but encouraged) */}
    <div className="reasoning-section">
      <label htmlFor="reasoning-field">
        <strong>Your Reasoning (Optional)</strong>
        <span className="helper-text">Document why you're making this decision. Helps with
future reviews.</span>
      </label>
      <textarea
        id="reasoning-field"
        rows="3"
        placeholder="E.g., 'Verified schema is production, data types match, and similar
mappings exist for related concepts.'"
      />
    </div>

```

```

    {/* Action Buttons */}
    <div className="action-buttons">
      <button
        className="btn btn-success"
        onClick={() => handleDecision('accepted')}
        disabled={!allCriticalChecksComplete(checklistState, explanation)}
      >
        ✓ Accept Mapping
      </button>

      <button
        className="btn btn-warning"
        onClick={() => handleDecision('modified')}
      >
        ✎ Modify & Accept
      </button>

      <button
        className="btn btn-danger"
        onClick={() => handleDecision('rejected')}
      >
        ✗ Reject Mapping
      </button>

      <button

```

```

        className="btn btn-secondary"
        onClick={() => handleDecision('deferred')}
      >
        ■■ Defer Decision
      </button>
    </div>

    { /* Helper Functions Info */ }
    <div className="helper-info">
      <p className="info-text">
        Need help? <a href="#" onClick={showValidationGuide}>View Validation Guide</a>
      </p>
    </div>

  </div>
);
};

// Helper functions
function getTotalChecklistItems(explanation) {
  return explanation.validation_checklist.reduce(
    (total, category) => total + category.items.length,
    0
  );
}

function allCriticalChecksComplete(checklistState, explanation) {
  // Find all critical checks (marked with ⚠️)
  const criticalChecks = [];
  explanation.validation_checklist.forEach((category, catIdx) => {
    category.items.forEach((item, itemIdx) => {
      if (item.startsWith('⚠️ ')) {
        criticalChecks.push(`check-${catIdx}-${itemIdx}`);
      }
    });
  });

  // All critical checks must be completed
  return criticalChecks.every(checkId => checklistState[checkId] === true);
}

function calculateComplexity(mapping) {
  // Calculate task complexity based on various factors
  let complexity = 0.5; // Base complexity

```

```

if (mapping.concerns && mapping.concerns.length > 0) {
  complexity += mapping.concerns.length * 0.1;
}

if (mapping.confidence < 0.7) {
  complexity += 0.2;
}

return Math.min(complexity, 1.0);
}

const ProgressBar = ({ value, color }) => (
  <div className="progress-bar-container">
    <div
      className={`progress-bar-fill progress-${color}`}
      style={{ width: `${value * 100}%` }}
    />
  </div>
);

export default MappingValidationInterface;
...

```

4.2 Tom Sawyer Enhanced Workbench Views

Due to length constraints, I'll provide the conceptual design for enhanced Tom Sawyer visualizations rather than full implementation.

Enhanced Knowledge Engineer Dashboard (Tom Sawyer Integration):

...

KNOWLEDGE ENGINEER WORKBENCH - Session Overview									
HUMAN PERFORMANCE					PENDING WORK				
Session Time: 3h 42m		High Priority: 23							
Decisions Made: 89		Medium: 71							
Current State:  MODERATE FATIGUE		Low: 33							
Validation Accuracy: 82% ↓		Est. Review Time: 8.5 hrs							
(Baseline: 91%)		Your Time Today: 3 hrs							
Trust Calibration:  OVER-TRUST		 Recommended:							

You're accepting 15% more than actual accuracy warrants		Focus on High Priority only today	
	[View Detailed Metrics]	[Filter by Domain]	
ACTIVE CONTEXT			
Domain Agents Active: Sales Operations, Order Management			
Subdomain Filter: Lead Management, Quote & Pricing			
Persona: Data Architect			
Ontology Version: v2.3.1 (Published: 2026-01-15)			
[Change Context]		[View Full Graph]	[Search Knowledge]
NEXT RECOMMENDED TASK			
Task: Resolve Conflict - CustomerAccount Mapping			
Complexity: ●●●○○ Moderate			
Priority: HIGH (Blocking 3 downstream systems)			
⚠ Given current fatigue level, this task is at upper limit of recommended complexity. Consider break first.			
[Start Task]		[Defer]	[Take 15-Min Break]
QUICK ACTIONS			
[Competency Self-Test]		[View My Performance Trends]	[Training]

...

 Enhanced Mapping Detail View:

 ...

MAPPING DETAIL - CustomerAccount → CRM_ACCOUNT_TBL		
CONFIDENCE ANALYSIS		
AI Confidence: 82%		
Category: HIGH (Top 25% of scores)		
⚠ CALIBRATION WARNING		
Historical accuracy at 82% confidence: 71%		
AI is overconfident by 11%. Treat as ~70% confidence.		
▶ Show Confidence Breakdown	📊 View Historical Trend	
GRAPH VISUALIZATION		
[Interactive Relationship Graph]		
<pre> graph LR Customer -- has --> CustomerAccount CustomerAccount -. "?" .-> CRM_ACCOUNT_TBL CustomerAccount -- similar --> CUST_ACCT_TBL_Alt[CUST_ACCT_TBL (Alt)] Customer -- stored_in --> CRM_CUSTOMER_TBL CRM_CUSTOMER_TBL -- fk --> CRM_ACCOUNT_TBL </pre>		
Legend: Green edge = Strong evidence Yellow = Moderate Red = Conflict Dotted = Proposed mapping Solid = Validated		
[Expand Graph] [Show Impact Analysis] [Export View]		
EVIDENCE & CONCERNS		
✓ String Match: Exact match on 'AccountID' field		
✓ Precedent: 15 similar mappings validated (92% accuracy)		
✓ Usage: Schema actively used (last modified: 2 days ago)		


```

        'manual_mapping_frequency_days': 14, # Every 2 weeks
        'knowledge_assessment_frequency_days': 30, # Monthly
        'conflict_simulation_frequency_days': 21, # Every 3 weeks
        'calibration_review_frequency_days': 7 # Weekly
    }

def check_maintenance_needed(self, engineer_id):
    """Check if engineer needs maintenance activities."""

    last_activities = self.db.get_last_maintenance_activities(engineer_id)

    needed_activities = []

    # Manual mapping exercise
    days_since_manual = (datetime.now() - last_activities.get('manual_mapping',
datetime.min)).days
    if days_since_manual >= self.maintenance_config['manual_mapping_frequency_days']:
        needed_activities.append({
            'type': 'MANUAL_MAPPING_EXERCISE',
            'priority': 'HIGH' if days_since_manual > 21 else 'MEDIUM',
            'reason': f'{days_since_manual} days since last manual mapping exercise',
            'estimated_time': 30, # minutes
            'description': 'Create a mapping from scratch to maintain manual skills'
        })

    # Knowledge assessment
    days_since_assessment = (datetime.now() - last_activities.get('knowledge_assessment',
datetime.min)).days
    if days_since_assessment >=
self.maintenance_config['knowledge_assessment_frequency_days']:
        needed_activities.append({
            'type': 'KNOWLEDGE_ASSESSMENT',
            'priority': 'MEDIUM',
            'reason': f'{days_since_assessment} days since last knowledge assessment',
            'estimated_time': 15,
            'description': 'Quiz on domain concepts and mapping principles'
        })

    # Conflict simulation
    days_since_simulation = (datetime.now() - last_activities.get('conflict_simulation',
datetime.min)).days
    if days_since_simulation >=
self.maintenance_config['conflict_simulation_frequency_days']:
        needed_activities.append({
            'type': 'CONFLICT_SIMULATION',
            'priority': 'HIGH' if days_since_simulation > 30 else 'MEDIUM',
            'reason': f'{days_since_simulation} days since conflict resolution practice',

```

```

        'estimated_time': 20,
        'description': 'Practice resolving complex mapping conflicts'
    })

    # Calibration review
    days_since_calibration = (datetime.now() - last_activities.get('calibration_review',
datetime.min)).days
    if days_since_calibration >= self.maintenance_config['calibration_review_frequency_days']:
        needed_activities.append({
            'type': 'CALIBRATION_REVIEW',
            'priority': 'LOW',
            'reason': 'Weekly trust calibration review',
            'estimated_time': 10,
            'description': 'Review your trust calibration metrics and adjust'
        })

    return {
        'needs_maintenance': len(needed_activities) > 0,
        'activities': sorted(needed_activities, key=lambda x: {'HIGH': 0, 'MEDIUM': 1, 'LOW': 2}
[x['priority']]),
        'total_estimated_time': sum(a['estimated_time'] for a in needed_activities)
    }

def generate_manual_mapping_exercise(self, engineer_id):
    """
    Generate a manual mapping exercise.
    Engineer must create mapping without AI assistance.
    """

    # Get engineer's expertise domain
    profile = self.db.get_engineer_profile(engineer_id)
    domain = profile.get('primary_domain', 'Sales Operations')

    # Select an unmapped concept
    unmapped_concepts = self.kg.get_unmapped_concepts(domain=domain, limit=10)

    # Select one with moderate difficulty
    concept = self._select_moderate_difficulty_concept(unmapped_concepts)

    # Get candidate schemas
    candidate_schemas = self.kg.get_candidate_schemas(concept['name'])

    return {
        'exercise_id': str(uuid.uuid4()),
        'type': 'MANUAL_MAPPING',
        'concept': concept,
        'candidate_schemas': candidate_schemas,

```

```
'instructions': f"""
```

```
    Create a mapping for concept '{concept['name']}' without AI assistance.
```

```
Steps:
```

1. Review the concept definition and properties
2. Examine candidate schemas (or search for others)
3. Select the best schema match
4. Document your reasoning
5. Assign a confidence score

```
    Your mapping will be compared to AI's suggestion and expert validation.
```

```
""",
```

```
'time_limit_minutes': 30,
```

```
'evaluation_criteria': [
```

```
    'Correctness of selected schema',
```

```
    'Quality of reasoning documentation',
```

```
    'Appropriate confidence assignment',
```

```
    'Consideration of alternatives'
```

```
]
```

```
}
```

```
def evaluate_manual_mapping(self, exercise_id, engineer_mapping):
```

```
    """
```

```
    Evaluate engineer's manual mapping exercise.
```

```
    Compare to AI suggestion and expert ground truth.
```

```
    """
```

```
    exercise = self.db.get_exercise(exercise_id)
```

```
    concept = exercise['concept']
```

```
    # Get AI suggestion for comparison
```

```
    ai_mapping = self.ai_mapper.suggest_mapping(concept['name'])
```

```
    # Get expert validation (if available)
```

```
    expert_mapping = self.db.get_expert_validation(concept['name'])
```

```
    # Evaluate correctness
```

```
    if expert_mapping:
```

```
        correct = (engineer_mapping['target_schema'] == expert_mapping['target_schema'])
```

```
    else:
```

```
        correct = None # Will be validated later
```

```
    # Evaluate reasoning quality
```

```
    reasoning_score = self._evaluate_reasoning_quality(
```

```
        engineer_mapping.get('reasoning', "")
```

```
)
```

```

# Evaluate confidence calibration
if correct is not None:
    confidence_error = abs(engineer_mapping['confidence'] - (1.0 if correct else 0.0))
    confidence_calibration = 1.0 - confidence_error
else:
    confidence_calibration = None

# Compare to AI
agreement_with_ai = (
    engineer_mapping['target_schema'] == ai_mapping['target_schema']
)

evaluation = {
    'exercise_id': exercise_id,
    'engineer_mapping': engineer_mapping,
    'ai_mapping': ai_mapping,
    'expert_mapping': expert_mapping,
    'correct': correct,
    'reasoning_quality_score': reasoning_score,
    'confidence_calibration': confidence_calibration,
    'agreement_with_ai': agreement_with_ai,
    'feedback': self._generate_feedback(
        engineer_mapping, ai_mapping, expert_mapping, reasoning_score
    )
}

# Store results
self.db.store_competency_assessment({
    'engineer_id': exercise['engineer_id'],
    'assessment_type': 'manual_mapping',
    'accuracy_score': 1.0 if correct else 0.0 if correct is not None else 0.5,
    'completeness_score': reasoning_score,
    'overall_score': self._calculate_overall_score(evaluation)
})

return evaluation

def _evaluate_reasoning_quality(self, reasoning_text):
    """
    Evaluate quality of reasoning documentation.
    Looks for key elements: evidence, alternatives considered, concerns.
    """

    if not reasoning_text or len(reasoning_text) < 50:
        return 0.2 # Minimal reasoning

    score = 0.3 # Base score for providing reasoning

```

```

# Evidence mentioned
if any(keyword in reasoning_text.lower() for keyword in
        ['because', 'evidence', 'match', 'similar', 'precedent']):
    score += 0.2

# Alternatives considered
if any(keyword in reasoning_text.lower() for keyword in
        ['alternative', 'considered', 'compared', 'other', 'instead']):
    score += 0.2

# Concerns or caveats mentioned
if any(keyword in reasoning_text.lower() for keyword in
        ['however', 'concern', 'caveat', 'limitation', 'uncertain']):
    score += 0.2

# Specific details (length and specificity)
if len(reasoning_text) > 200:
    score += 0.1

return min(score, 1.0)

def _generate_feedback(self, engineer_mapping, ai_mapping, expert_mapping,
reasoning_score):
    """Generate constructive feedback on manual mapping exercise."""

    feedback = []

    # Correctness feedback
    if expert_mapping:
        if engineer_mapping['target_schema'] == expert_mapping['target_schema']:
            feedback.append({
                'type': 'SUCCESS',
                'message': f"✓ Correct! '{engineer_mapping['target_schema']}' is the expert-
validated mapping."
            })
        else:
            feedback.append({
                'type': 'CORRECTION',
                'message': f"The expert-validated mapping is '{expert_mapping['target_schema']}',
not '{engineer_mapping['target_schema']}'.",
                'learning_point': f"Review: {expert_mapping.get('reasoning', 'No reasoning
provided'))}"
            })

    # AI comparison

```

```

        if engineer_mapping['target_schema'] == ai_mapping['target_schema']:
            feedback.append({
                'type': 'INFO',
                'message': f"You agreed with AI suggestion ({ai_mapping['confidence']:.0%}
confidence)"
            })
        else:
            feedback.append({
                'type': 'INFO',
                'message': f"You selected '{engineer_mapping['target_schema']}' while AI suggested
'{ai_mapping['target_schema']}' ({ai_mapping['confidence']:.0%} confidence)"
            })

    # Reasoning quality feedback
    if reasoning_score < 0.5:
        feedback.append({
            'type': 'IMPROVEMENT',
            'message': 'Reasoning documentation was minimal. Include: (1) Evidence supporting
your choice, (2) Alternatives you considered, (3) Any concerns or caveats.'
        })
    elif reasoning_score < 0.7:
        feedback.append({
            'type': 'IMPROVEMENT',
            'message': 'Good reasoning, but could be more comprehensive. Consider
documenting alternatives you ruled out and why.'
        })
    else:
        feedback.append({
            'type': 'SUCCESS',
            'message': '✓ Excellent reasoning documentation. Clear evidence, alternatives, and
concerns.'
        })

    return feedback

def _calculate_overall_score(self, evaluation):
    """Calculate overall competency score from evaluation."""

    if evaluation['correct'] is None:
        # No ground truth yet
        return 0.5

    accuracy = 1.0 if evaluation['correct'] else 0.0
    reasoning = evaluation['reasoning_quality_score']

    # Weighted average

```

```
return (accuracy * 0.6) + (reasoning * 0.4)
```

5.2 Knowledge Assessment Quiz Generator

```
class KnowledgeAssessmentGenerator:
    """
    Generate domain knowledge quizzes to ensure engineers maintain
    conceptual understanding despite automation.
    """

    def __init__(self, knowledge_graph):
        self.kg = knowledge_graph

    def generate_quiz(self, engineer_id, domain, difficulty='medium'):
        """Generate a knowledge assessment quiz."""

        # Get engineer's performance history
        history = self.db.get_assessment_history(engineer_id, 'knowledge_quiz')

        # Generate questions targeting weak areas
        questions = []

        # Question Type 1: Concept Definitions
        questions.extend(self._generate_definition_questions(domain, count=3))

        # Question Type 2: Relationship Understanding
        questions.extend(self._generate_relationship_questions(domain, count=3))

        # Question Type 3: Mapping Scenarios
        questions.extend(self._generate_scenario_questions(domain, count=2))

        # Question Type 4: Error Detection
        questions.extend(self._generate_error_detection_questions(domain, count=2))

        return {
            'quiz_id': str(uuid.uuid4()),
            'engineer_id': engineer_id,
            'domain': domain,
            'difficulty': difficulty,
            'questions': questions,
            'time_limit_minutes': 15,
            'passing_score': 0.7
        }

    def _generate_definition_questions(self, domain, count):
```

```

"""Generate questions about concept definitions."""

concepts = self.kg.get_domain_concepts(domain, limit=count * 2)
selected = random.sample(concepts, count)

questions = []
for concept in selected:
    # Get definition
    definition = concept['definition']
# Get similar concepts for distractors
    similar = self.kg.get_similar_concepts(concept['name'], limit=3)

    questions.append({
        'type': 'MULTIPLE_CHOICE',
        'question': f"What is the correct definition of '{concept['name']}'?",
        'options': [
            {'text': definition, 'correct': True},
            {'text': similar[0]['definition'], 'correct': False},
            {'text': similar[1]['definition'], 'correct': False},
            {'text': similar[2]['definition'], 'correct': False}
        ],
        'explanation': f"'{concept['name']}' is defined as: {definition}"
    })

return questions

def _generate_scenario_questions(self, domain, count):
    """Generate scenario-based questions about mapping decisions."""

    scenarios = [
        {
            'context': "You're mapping the concept 'OrderTotal' to a schema. You find two candidate tables: 'ORDER_SUMMARY' (confidence: 0.85) and 'ORDER_DETAILS' (confidence: 0.82). The SUMMARY table has 'total_amount' field, while DETAILS has 'line_item_total' field.",
            'question': "Which schema should you select and why?",
            'correct_answer': "ORDER_SUMMARY because 'total_amount' directly represents the order total, while 'line_item_total' is a component that would need aggregation. The confidence scores are similar, so semantic fit is the deciding factor.",
            'distractor_1': "ORDER_DETAILS because it has slightly lower confidence, making it safer to choose the more conservative option.",
            'distractor_2': "Both are equally valid; flip a coin.",
            'distractor_3': "Neither; the confidence gap is too small to make a reliable decision."
        },
        {
            'context': "You're reviewing an AI-suggested mapping with 0.91 confidence. However, you notice the target schema was last modified 3 years ago and has 'DEPRECATED' in the description field.",

```

```

        'question': "What should you do?",
        'correct_answer': "Reject the mapping despite high confidence. The schema is
deprecated and shouldn't be used, regardless of AI confidence. Flag for investigation into
replacement schema.",
        'distractor_1': "Accept the mapping since confidence is very high (>0.90).",
        'distractor_2': "Lower the confidence score to 0.70 and then accept.",
        'distractor_3': "Accept but add a note about deprecation for future review."
    }
]

return [{
    'type': 'SCENARIO',
    'context': s['context'],
    'question': s['question'],
    'options': [
        {'text': s['correct_answer'], 'correct': True},
        {'text': s['distractor_1'], 'correct': False},
        {'text': s['distractor_2'], 'correct': False},
        {'text': s['distractor_3'], 'correct': False}
    ],
    'explanation': s['correct_answer']
} for s in scenarios[:count]]
---
```

Part 6: Organizational Safeguards

6.1 Production Pressure Monitoring

```

```python
class ProductionPressureMonitor:
 """
 Detect when production pressure is compromising safety and quality.
 Provides organizational visibility into pressure vs. quality tradeoffs.
 """

 def __init__(self, performance_db, workload_db):
 self.performance_db = performance_db
 self.workload_db = workload_db
 self.pressure_thresholds = {
 'decision_speed_increase': 2.5, # 2.5x faster than baseline
 'accuracy_decline_acceptable': 0.10, # 10% drop
 'override_rate_warning': 0.20, # 20% of safety alerts overridden
 'consecutive_long_sessions': 3 # 3+ sessions over 6 hours
 }

 def detect_production_pressure(self, engineer_id=None, team_id=None):
 """
 Detect signs of production pressure compromising quality.

```

Can analyze individual engineer or entire team.

"""

time\_window\_days = 14 # Analyze last 2 weeks

if engineer\_id:

    indicators = self.\_analyze\_individual(engineer\_id, time\_window\_days)

elif team\_id:

    indicators = self.\_analyze\_team(team\_id, time\_window\_days)

else:

    indicators = self.\_analyze\_organization(time\_window\_days)

# Calculate overall pressure score

pressure\_score = self.\_calculate\_pressure\_score(indicators)

# Classify pressure level

if pressure\_score > 0.7:

    pressure\_level = 'CRITICAL'

    alert\_severity = 'HIGH'

elif pressure\_score > 0.5:

    pressure\_level = 'HIGH'

    alert\_severity = 'MEDIUM'

elif pressure\_score > 0.3:

    pressure\_level = 'MODERATE'

    alert\_severity = 'LOW'

else:

    pressure\_level = 'NORMAL'

    alert\_severity = 'NONE'

return {

    'pressure\_level': pressure\_level,

    'pressure\_score': pressure\_score,

    'alert\_severity': alert\_severity,

    'indicators': indicators,

    'recommendations': self.\_generate\_pressure\_recommendations(indicators,  
pressure\_level),

    'requires\_management\_action': pressure\_level in ['CRITICAL', 'HIGH']

}

def \_analyze\_individual(self, engineer\_id, time\_window\_days):

    """Analyze individual engineer for pressure indicators."""

    baseline = self.performance\_db.get\_baseline\_metrics(engineer\_id)

    recent = self.performance\_db.get\_recent\_metrics(engineer\_id, days=time\_window\_days)

    indicators = {}

```

Indicator 1: Dramatic speed increase (rushing)
if recent['avg_decision_time'] < baseline['avg_decision_time'] /
self.pressure_thresholds['decision_speed_increase']:
 indicators['rushing_detected'] = {
 'severity': 'HIGH',
 'current_speed': recent['avg_decision_time'],
 'baseline_speed': baseline['avg_decision_time'],
 'speed_ratio': baseline['avg_decision_time'] / recent['avg_decision_time'],
 'interpretation': f"Making decisions {baseline['avg_decision_time'] /
recent['avg_decision_time']:.1f}x faster than baseline. Likely rushing due to workload pressure."
 }

```

```

Indicator 2: Accuracy decline
accuracy_decline = baseline['accuracy'] - recent['accuracy']
if accuracy_decline > self.pressure_thresholds['accuracy_decline_acceptable']:
 indicators['accuracy_decline'] = {
 'severity': 'HIGH' if accuracy_decline > 0.15 else 'MEDIUM',
 'current_accuracy': recent['accuracy'],
 'baseline_accuracy': baseline['accuracy'],
 'decline_amount': accuracy_decline,
 'interpretation': f"Accuracy declined {accuracy_decline:.1%}. May indicate quality
compromised for speed."
 }

```

```

Indicator 3: Safety alert overrides
alerts = self.performance_db.get_recent_alerts(engineer_id, days=time_window_days)
overridden = [a for a in alerts if a['status'] == 'overridden']
override_rate = len(overridden) / len(alerts) if alerts else 0

if override_rate > self.pressure_thresholds['override_rate_warning']:
 indicators['excessive_overrides'] = {
 'severity': 'HIGH',
 'total_alerts': len(alerts),
 'overridden_count': len(overridden),
 'override_rate': override_rate,
 'interpretation': f"Overriding {override_rate:.0%} of safety alerts. Suggests pressure to
'push through' warnings."
 }

```

```

Indicator 4: Consecutive long sessions
sessions = self.performance_db.get_recent_sessions(engineer_id,
days=time_window_days)
long_sessions = [s for s in sessions if s['duration_minutes'] > 360] # >6 hours

```

```

Check for consecutive long sessions
consecutive_long = 0
max_consecutive = 0

```

```

for i, session in enumerate(sorted(sessions, key=lambda s: s['start_time'])):
 if session['duration_minutes'] > 360:
 consecutive_long += 1
 max_consecutive = max(max_consecutive, consecutive_long)
 else:
 consecutive_long = 0

if max_consecutive >= self.pressure_thresholds['consecutive_long_sessions']:
 indicators['burnout_risk'] = {
 'severity': 'CRITICAL',
 'consecutive_long_sessions': max_consecutive,
 'total_long_sessions': len(long_sessions),
 'interpretation': f"{max_consecutive} consecutive sessions over 6 hours. High burnout
risk."
 }

Indicator 5: Weekend/evening work
off_hours_sessions = [s for s in sessions if self._is_off_hours(s['start_time'])]
if len(off_hours_sessions) / len(sessions) > 0.3 if sessions else False:
 indicators['off_hours_work'] = {
 'severity': 'MEDIUM',
 'off_hours_percentage': len(off_hours_sessions) / len(sessions),
 'interpretation': f"{len(off_hours_sessions) / len(sessions):.0%} of sessions during
evenings/weekends. Work-life balance concern."
 }

return indicators

def _analyze_team(self, team_id, time_window_days):
 """Analyze entire team for systemic pressure indicators."""

 team_members = self.workload_db.get_team_members(team_id)

 # Aggregate individual indicators
 all_indicators = []
 for member_id in team_members:
 individual = self._analyze_individual(member_id, time_window_days)
 if individual:
 all_indicators.append(individual)

 # Calculate team-wide metrics
 team_indicators = {}

 # How many team members show pressure signs?
 members_with_rushing = sum(1 for ind in all_indicators if 'rushing_detected' in ind)
 members_with_accuracy_decline = sum(1 for ind in all_indicators if 'accuracy_decline' in
ind)

```

```

members_with_burnout_risk = sum(1 for ind in all_indicators if 'burnout_risk' in ind)

total_members = len(team_members)

if members_with_rushing / total_members > 0.5:
 team_indicators['systemic_rushing'] = {
 'severity': 'CRITICAL',
 'affected_members': members_with_rushing,
 'total_members': total_members,
 'percentage': members_with_rushing / total_members,
 'interpretation': f"{members_with_rushing / total_members:.0%} of team showing
rushing behavior. Systemic workload issue."
 }

 if members_with_burnout_risk > 0:
 team_indicators['team_burnout_risk'] = {
 'severity': 'CRITICAL',
 'affected_members': members_with_burnout_risk,
 'interpretation': f"{members_with_burnout_risk} team members at burnout risk.
Immediate intervention needed."
 }

 # Workload distribution analysis
 workload_variance = self._analyze_workload_distribution(team_members,
time_window_days)
 if workload_variance > 0.3: # High variance in workload
 team_indicators['uneven_workload'] = {
 'severity': 'MEDIUM',
 'variance': workload_variance,
 'interpretation': 'Workload unevenly distributed. Some members overloaded while
others underutilized.'
 }

 return team_indicators

def _calculate_pressure_score(self, indicators):
 """Calculate overall pressure score from indicators."""

 if not indicators:
 return 0.0

 severity_weights = {
 'CRITICAL': 1.0,
 'HIGH': 0.7,
 'MEDIUM': 0.4,
 'LOW': 0.2
 }

```

```

scores = [
 severity_weights.get(indicator['severity'], 0)
 for indicator in indicators.values()
]

return sum(scores) / len(scores) if scores else 0.0

def _generate_pressure_recommendations(self, indicators, pressure_level):
 """Generate recommendations to address production pressure."""

 recommendations = []

 if pressure_level == 'CRITICAL':
 recommendations.append({
 'type': 'IMMEDIATE_ACTION',
 'action': 'HALT_NON_CRITICAL_WORK',
 'description': 'Stop all non-critical validation work immediately. Focus only on high-
priority items until team capacity restored.',
 'authority_required': 'MANAGEMENT'
 })

 if 'rushing_detected' in indicators or 'systemic_rushing' in indicators:
 recommendations.append({
 'type': 'WORKLOAD_ADJUSTMENT',
 'action': 'REDUCE_DAILY_TARGETS',
 'description': 'Reduce daily validation targets by 30-50% to allow thorough review
without rushing.',
 'authority_required': 'MANAGEMENT'
 })

 if 'burnout_risk' in indicators or 'team_burnout_risk' in indicators:
 recommendations.append({
 'type': 'MANDATORY_TIME_OFF',
 'action': 'ENFORCE_BREAKS',
 'description': 'Enforce mandatory time off for affected engineers. Minimum 3
consecutive days.',
 'authority_required': 'MANAGEMENT'
 })
 recommendations.append({
 'type': 'STAFFING',
 'action': 'HIRE_ADDITIONAL_RESOURCES',
 'description': 'Current workload exceeds sustainable capacity. Hire additional
knowledge engineers or contractors.',
 'authority_required': 'EXECUTIVE'
 })

```

```

 if 'excessive_overrides' in indicators:
 recommendations.append({
 'type': 'SAFETY_CULTURE',
 'action': 'REINFORCE_SAFETY_PRIORITY',
 'description': 'Communicate that quality and safety take precedence over throughput.
Engineers have authority to halt for quality concerns.',
 'authority_required': 'MANAGEMENT'
 })

 if 'accuracy_decline' in indicators:
 recommendations.append({
 'type': 'QUALITY_INTERVENTION',
 'action': 'PEER_REVIEW_REQUIRED',
 'description': 'Implement mandatory peer review for all validations until accuracy
returns to baseline.',
 'authority_required': 'TEAM_LEAD'
 })

 return recommendations

def _is_off_hours(self, timestamp):
 """Check if timestamp is outside normal work hours."""

 hour = timestamp.hour
 is_weekday = timestamp.weekday() < 5 # Monday = 0, Friday = 4

 if not is_weekday:
 return True # Weekend

 if hour < 8 or hour > 18:
 return True # Before 8am or after 6pm

 return False
...

```

## ## Part 7: Implementation Roadmap

### ### 7.1 Phased Implementation

#### \*\*Phase 1: Foundation (Months 1-2)\*\*

- Implement Human\_Performance\_Metrics database schema
- Deploy ValidationQualityTracker service
- Add basic fatigue detection to existing workflows
- Integrate performance alerts into Tom Sawyer Workbench

**\*\*Deliverables:\*\***

- Performance monitoring database operational
- Engineers receive alerts when fatigue/rushing detected
- Baseline metrics established for all knowledge engineers

**\*\*Phase 2: Trust Calibration (Months 2-3)\*\***

- Deploy TrustCalibrationMonitor service
- Implement confidence explainability service
- Enhanced mapping validation interface with confidence breakdown
- Personal trust calibration dashboards

**\*\*Deliverables:\*\***

- Engineers see confidence explanations for all AI suggestions
- Trust calibration warnings displayed when over/under-trusting
- Historical calibration data tracked

**\*\*Phase 3: Cognitive Support (Months 3-4)\*\***

- Deploy CognitiveWorkloadManager service
- Implement intelligent task assignment
- Break recommendations and enforcement
- Enhanced Tom Sawyer dashboards with human metrics

**\*\*Deliverables:\*\***

- Workload balanced across team based on cognitive capacity
- Automated break recommendations
- Real-time cognitive load indicators

**\*\*Phase 4: Competency Maintenance (Months 4-6)\*\***

- Deploy CompetencyMaintenanceScheduler
- Implement manual mapping exercises
- Knowledge assessment quizzes
- Skill tracking and degradation alerts

**\*\*Deliverables:\*\***

- Periodic competency exercises for all engineers
- Skill degradation detected and remediated
- Training automatically scheduled when needed

**\*\*Phase 5: Organizational Safeguards (Months 6-7)\*\***

- Deploy ProductionPressureMonitor
- Management dashboards for team health
- Safety culture metrics
- Override pattern analysis

**\*\*Deliverables:\*\***

- Management visibility into quality vs. pressure tradeoffs
- Early warning system for burnout

- Safety culture reinforcement mechanisms

**\*\*Phase 6: Full Integration & Optimization (Months 7-8)\*\***

- Integration testing across all services
- Performance optimization
- User training and change management
- Continuous improvement framework

**\*\*Deliverables:\*\***

- Fully integrated Human-AI Teaming system
- Comprehensive training for all users
- Ongoing monitoring and optimization process

**### 7.2 Success Metrics**

**\*\*Human Performance Metrics:\*\***

- Validation accuracy maintained above 90% across all sessions
- Decision quality does not degrade over session duration
- Zero critical safety overrides due to production pressure
- Engineer burnout indicators below 0.3 threshold

**\*\*Trust Calibration Metrics:\*\***

- Trust miscalibration reduced to <5% delta from accuracy
- Over-trust incidents reduced by 80% from baseline
- Engineers adjust behavior based on calibration feedback

**\*\*System Effectiveness Metrics:\*\***

- AI Circuit Breaker prevents 100% of high-risk actions
- Manual intervention success rate >95% (when breaker trips, humans make correct decision)
- Competency scores maintained within 10% of baseline despite automation

**\*\*Organizational Metrics:\*\***

- Production pressure indicators remain in "NORMAL" range
- Team burnout risk <10% of members
- Safety culture scores >4.0/5.0 on surveys

---

**## Part 8: Conclusion**

**### The Human-AI Partnership Imperative**

This framework addresses the fundamental gap in AI governance: **\*\*maintaining effective human oversight as automation improves.\*\***

The Semantic Governance architecture now has:

✓ **\*\*Technical Excellence\*\*** (Documents 1-6)

- World-class semantic infrastructure
- Sophisticated AI agents
- Comprehensive process governance
- Advanced visualization tools

✓ **\*\*Human Factors Engineering\*\*** (Document 7 - This Framework)

- Performance monitoring that detects degradation
- Trust calibration that prevents complacency
- Cognitive support that maintains effectiveness
- Competency maintenance that preserves skills
- Organizational safeguards that protect safety culture

### The Difference This Makes

**\*\*Without Human Factors Framework:\*\***

Week 1: AI 65% accurate, Humans 95% oversight → System 95% reliable  
Week 52: AI 92% accurate, Humans 70% oversight → System 85% reliable

Result: Better AI → Worse overall system

**\*\*With Human Factors Framework:\*\***

Week 1: AI 65% accurate, Humans 95% oversight → System 95% reliable  
Week 52: AI 92% accurate, Humans 94% oversight → System 99% reliable

Result: Better AI + Sustained human performance → Better system